

Intermediate SQL

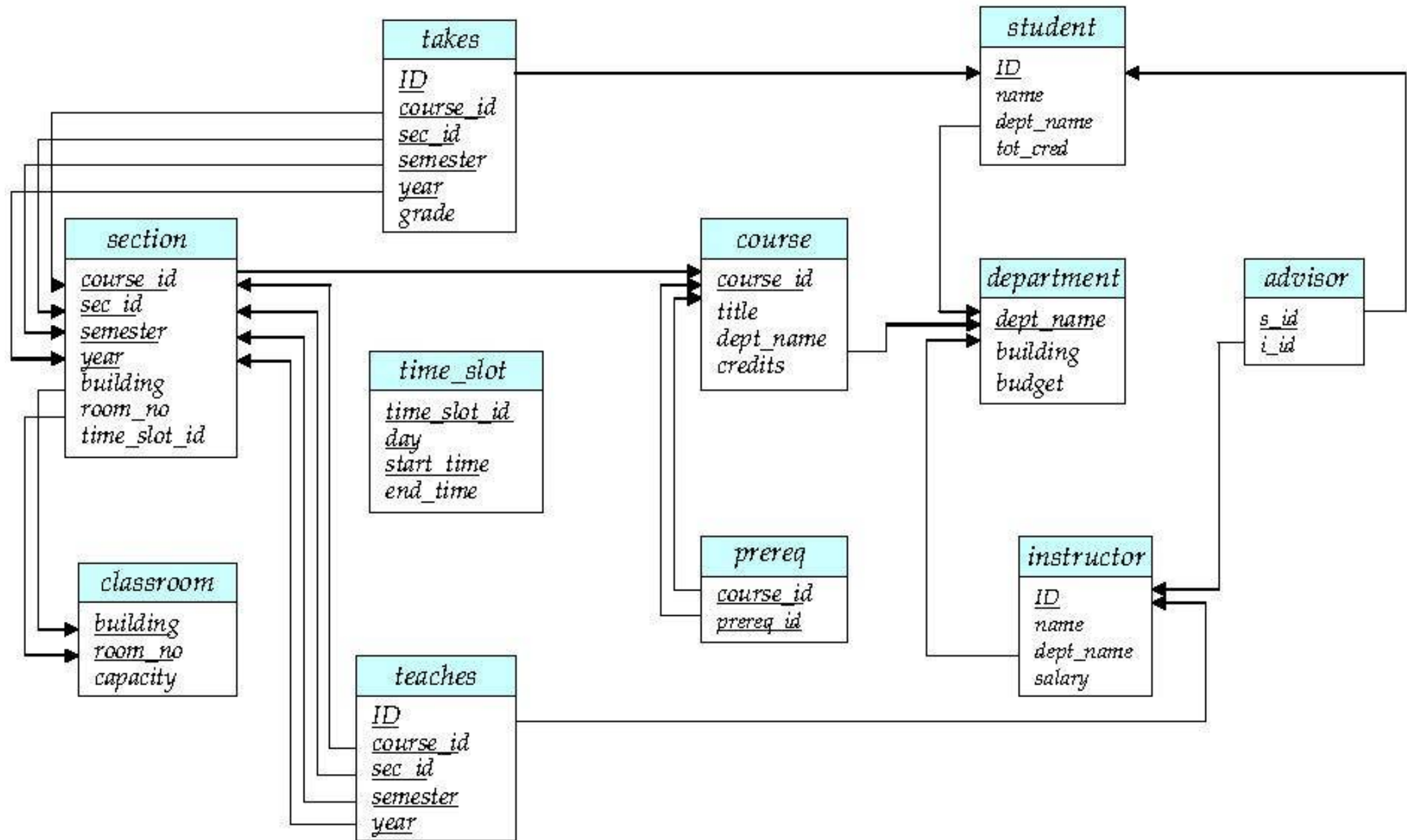
— Chapter 4

2025 年 10
月

School of Computer Science



北京邮电大学



4.1 Joined Expressions

- **Def: Join operations** take **two** relations and return as a result **another** relation.
- **Def: Join operation** is a **Cartesian product** requires that **tuples** in two relations **match**.
 - The attributes are in the result of the join
 - Cartesian product $\bowtie$ + select σ
- **Def: Join condition** – defines **which tuples** in two relations **match**, and **what attributes** are present as the result.
- **Def: Join type** – defines **how tuples** in each relation that **do not match** any tuple in the other relation are treated.

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join Conditions</i>
natural
on <predicate>
using ($A_1, A_1, \dots, A_n$)

(1) Natural Join operations

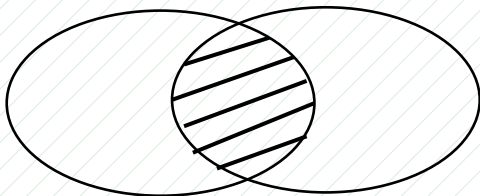
- Two relations

- course*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- prereq*

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101



- select ***

- from** *course* **natural join** *prereq*

- select ***

- from** *course* **join** *prereq*

- on** *course.course_id*=*prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

course **natural join** *prereq* **on** *course_id*

➤ *course* information is missing for CS-315

➤ *prereq* information is missing for CS-437

(2) Outer Join

- **Def:** Extension of the **join** operation avoids **loss** of information.
- **Computes** the join, and then **adds** tuples from one relation that **does not match** tuples in another relation **to** the result of the **join**.

Uses the **null** values.

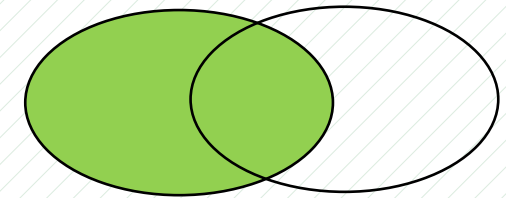
- **1. Left Outer Join**

- *course* **natural left outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

- **select ***

from *course* **natural left outer join** *prereq*



Outer Join

■ 2. Right Outer Join

- *course* **natural right outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- **select ***

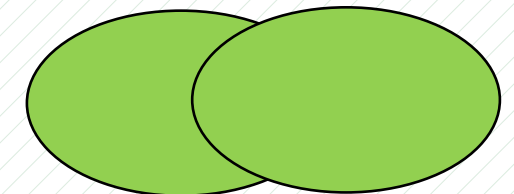
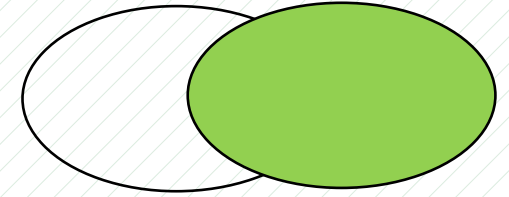
from *course* **natural right outer join** *prereq*

■ 3. Full Outer Join

- **select ***

from *course* **natural full outer join** *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



Joined Relations – Examples

- Difference between *inner join* and *natural join*

- *course* **inner join** *prereq* **on**
course.course_id = prereq.course_id

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

- *course* **left outer join** *prereq* **on**
course.course_id = prereq.course_id

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	<i>null</i>	<i>null</i>

Joined Relations – Examples

- ❑ **course natural right outer join prereq**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<u><i>prereq_id</i></u>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

- ❑ **course full outer join prereq using (course_id)**

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<u><i>prereq_id</i></u>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

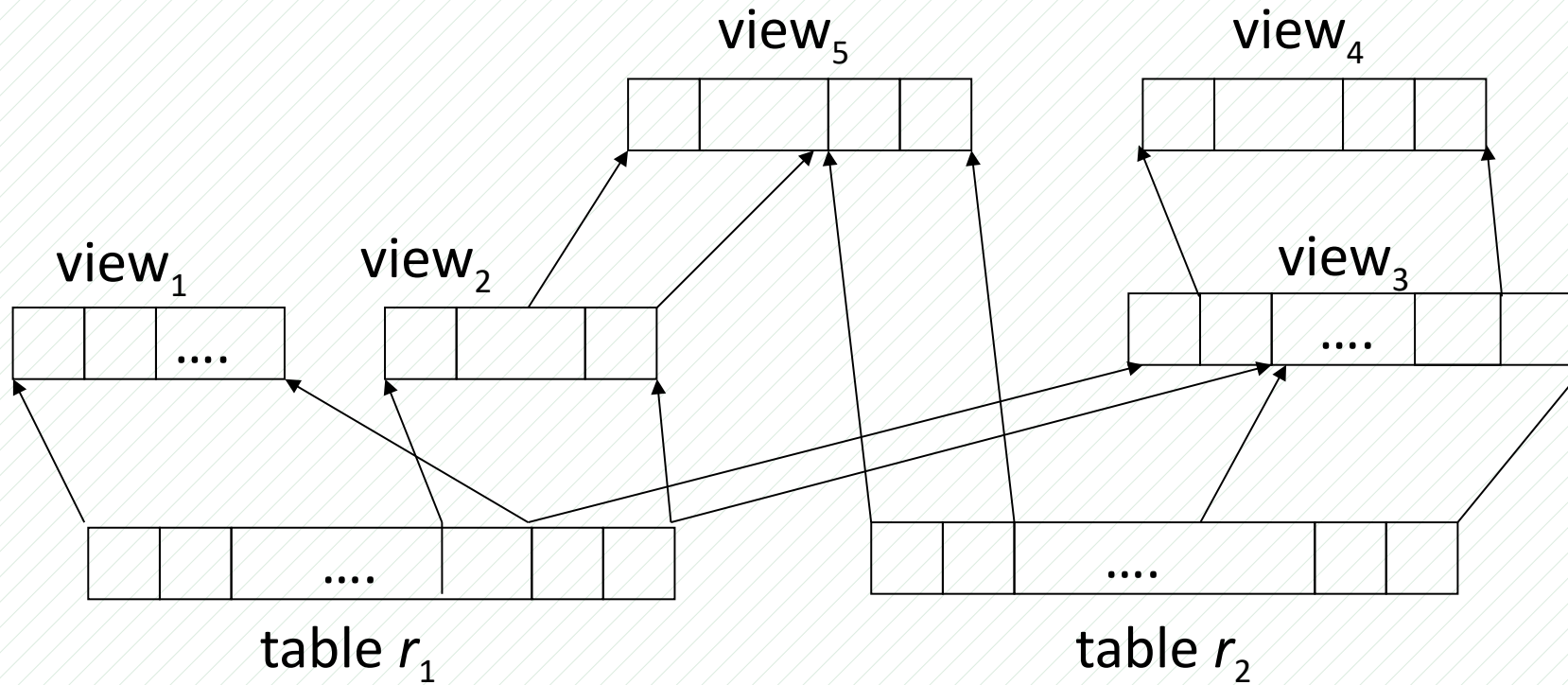
4.2 Views

- **Not desirable** for all users to see the **entire** logical model.
 - i.e., Someone needs to know an instructor name and department, **but not the salary.**
- See a relation

```
select ID, name, dept_name
from instructor
```
- **Def:** any relation that is **not** of the **conceptual** model ,
but is made **visible** to a user as a **virtual relation**.
- A **view hides** certain data from view of certain users

Views Definition

- **Note** : Views = **projection** on **one or more** relations



Views as Virtual Table

- **Note: View** is known as **virtual relation/table**
 - **only** the **definition of view** itself is stored in DBS, the **tuples** of the view is stored in relations
 - **evaluation** of view is reduced to evaluation of **relation-algebra expression** that defines the view
- **Note:** differences between the ***with*** clause and the ***create view*** clause
 - **with clause** creates a **temporal table** to store the query results, the created table will be **cancelled** after SQL execution ends
- **Def: Materialized views** (实体化 / 物化视图)
 - both **definition** and **data** of views are stored in DBS

View Definition

- **Def: View** is defined using the **create view** statement
create view *V* as <query expression >
where <query expression> is SQL expression.
The view name is represented by ***V***.
- The view name can be used to refer to the **virtual**.
- View definition is **not the same as** creating a new relation
 - a view definition causes the saving of an expression

Example Views

- A view of instructors without their salary

create view *faculty* as

select *ID, name, dept_name*

from *instructor*

- Find all instructors in the *Biology* department

select *name*

from *faculty*

where *dept_name* = 'Biology'

- A view of department salary totals

create view *departments_total_salary* (*dept_name, total_salary*) as

select *dept_name, sum (salary)*

from *instructor*

group by *dept_name*

Views Defined Using Other Views

- 1. create view *physics_fall_2019* as
select *course.course_id, sec_id, building, room_number*
from *course, section*
where *course.course_id = section.course_id*
and *course.dept_name = 'Physics'*
and *section.semester = 'Fall'*
and *section.year = '2019'*;
- 2. create view *physics_fall_2019_watson* as
select *course_id, room_number*
from *physics_fall_2019*
where *building= 'Watson'*;

Views Defined Using Other Views

Note: One view may be used in the expression defining another view

- **Def:** A view relation v_1 is said to **depend directly** on a view relation v_2 if v_2 is used in the expression defining v_1
- **Def:** A view relation v_1 is said to **depend on** view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- **Def:** A view relation v is said to be **recursive** if it depends on itself

View Expansion

- **Expand use** of a view in a query/another view

```
create view physics_fall_2009_watson as  
(select course_id, room_number  
from (select course.course_id, building, room_number  
      from course, section  
      where course.course_id = section.course_id  
           and course.dept_name = 'Physics'  
           and section.semester = 'Fall'  
           and section.year = '2009')  
where building = 'Watson';
```

Update of a View

- **Add** a new tuple to *faculty* view

insert into *faculty* values ('30765', 'Green', 'Music');

- This insertion must be represented by
insertion of the tuple

('30765', 'Green', 'Music', **null**)

into the *instructor* relation

Materialized Views

- **Materializing a view**
 - ▣ create a physical table containing all the tuples in the result of the query defined view
- ▣ If relations used in the query are **updated**, the materialized view result becomes **out of date**
 - ▣ need to **maintain** the view, by updating the view whenever the underlying relations are updated.

4.3 Transactions

- **Def.:** A **transaction** consists of a **sequence** of query and/or update statements and it is a **unit** of work
- **Note : ACID** properties
 - Atomicity, Consistency, Isolation, Durability
- **Atomic** transaction
 - either fully executed or rolled back as if never occurred
- **Isolation** from concurrent transactions
- **Note:** A transaction **begins implicitly** when an statement is executed.
ends with statements:
 - **Commit work.** The updates become **permanent**.
 - **Rollback work.** All the updates performed are **undone**.

bank_account_transfer:
Initially, $A=1000$,
 $B=2000$,
 $A+B=3000$
transfer: $A=A-50=950$
 $B=B+50=2050$
 $A+B=3000$

```
DECLARE @transfer_name varchar(10)          /* 事务变量定义 */
SET @transfer_name = 'I-transfer-from-A-to-B' /* 事务命名 */
BEGIN TRANSACTION @transfer_name             /* 事务开始 */
USE ACCOUNT                                  /* 打开数据库 ACCOUNT*/
GO                                             /* 将上述批 SQL 语句提交 SQL Server
UPDATE A                                     /* 修改 A 帐户 */
    SET balance = balance - 50
    WHERE branch_name= 'Brooklyn'
UPDATE B                                     /* 修改 B 帐户 */
    SET balance = balance + 50
    WHERE branch_name= 'Brooklyn'
GO
COMMIT TRANSACTION @transfer_name            /* 事务提交 */
GO
```

4.4 Integrity Constraints

- **Def: Integrity constraints** guard against **accidental damage** to DB, ensuring that the **authorized changes** to DB **do not** result in **loss** of data **consistency**
 - an instructor name can not be **null**
 - **no** two instructors can have the same instructor *ID*
 - every **department name** in *course* relation must have a matching department name in *department* relation

Classification of integrity constraints

	attribute-level	tuple-level	relation-level
static	■ data type ■ data format, ■ domain constraints, e.g.1 ■ null value	constraints among attributes values ➤ e.g.2 ➤ mapping cardinality constraints	■ entity integrity e.g.3 ■ referential integrity ■ functional dependency ➤ e.g.6
dynamic	constraints on updating of attribute values or attribute definition ➤ e.g.4	constraints among attributes values ➤ e.g.5	transaction constraint: atomy, consistency, isolation, durability

return

Examples : Integrity Constraints

- E.g. 1 the salary of manager should not be lower than \$1000 in *Employee*
- E.g. 2 table T (x, y, z), $z = x + y$, z is a **derived** attributes from x and y.
- E.g. 3 the *student#* for table *student* should not be null
- E.g. 4 the age of students should only be added
- E.g. 5 when employee tuples is modified, $\text{new.sal} > \text{old.sal} + 0.5 * \text{age}$
- E.g. 6 statistical constraints
 - ▣ in table, the *salary* of manager should be four times more than that of workers

Constraints on Single Relation

- **Relation** is defined using the **create table** command

```
create table r (A1 D1, A2 D2, ..., An Dn,  
              (integrity-constraint1), ... ,  
              (integrity-constraintk))
```

- *r* is the name of the relation
- each *A_i* is an attribute name in schema of relation *r*
- *D_i* is the data type of values in domain of attribute *A_i*
- **Note:** Integrity constraints include
 - primary key
 - **not null**
 - unique
 - check (*P*), where *P* is a predicate
- **not null**
 - declare name and budget to be not null
name **varchar(20) not null**
budget **numeric(12,2) not null**

Constraints on Single Relation (cont.)

- **Def: Unique constraint** is as follows

unique ($A_1, A_2, \dots, A_m$)

- attributes $A_1, A_2, \dots, A_m$ form a **candidate key**
- **candidate keys are permitted to be null** (in contrast to **primary keys**)
- e.g. **create table** *customer*
 (*customer-id* char(15)
 customer-name char(15)
 customer-city char(30),
 primary key (*customer-id*)
 unique (*customer-name*)

Constraints on Single Relation (cont.)

- The **check** clause is applied to **relation declaration** as well as **domain declaration**
 - ▣ **check** (P), where P is a predicate
 - ▣ e.g. ensure that semester in {fall, winter, spring or summer}

```
create table section (  
    course_id      varchar (8),  
    sec_id         varchar (8),  
    semester       varchar (6),  
    year           numeric (4,0),  
    room_number    varchar (7),  
    primary key   (course_id, sec_id, semester, year),  
    check         (semester in ('Fall', 'Winter', 'Spring', 'Summer'))  
);
```

Referential Integrity

- **Referential Integrity (关联 / 参照完整性)**
 - **ensures** that a value that **appears in one** relation for given attributes (foreign key) **also appears for the** attributes (primary key) in another relation
 - If **Biology** is a department name appearing in the tuples in **instructor** relation, then there exists a tuple in **department** relation for **Biology**

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	Math	null

instructor

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
null	Painter	null

department

where is **Math**?

Referential Integrity (cont.)

■ Def: referential integrity dependencies

□ Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys K_1 and K_2 respectively,

– e.g. r_1 : **department**(dept_name, building, budgt)

r_2 : **course**(course_id, title, dept_name, credits),

□ the subset α of R_2 (dept_name) is a **foreign key** (from r_2 , **course**)
referencing K_1 in relation r_1 (dept_name in **department**),
if for every t_2 in r_2 , there must be a tuple t_1 in r_1 such that

$$t_1[K_1] = t_2[\alpha]$$

□ **Note:** $\alpha = K_1 \subseteq R_2$



<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

r_1

Figure A.4 The *department* relation.

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

r_2

Figure A.5 The *course* relation.

return

Referential Integrity (cont.)

- ▣ **Def:** referential integrity constraint called **subset dependency**

since it can be written as

$$\Pi_{\alpha}(r_2) \subseteq \Pi_{K_1}(r_1)$$

- e.g. $\Pi_{dept_name}(course) \subseteq \Pi_{dept_name}(department)$

- ▣ **foreign key** α of table r_2 **references**

primary key attributes K_1 of the *referenced table* r_1

- **Note:** A **good** DB design should ensure that any relation schema R_2 can only reference other relation schema R_1 through its **foreign key**

Referential Integrity (cont.)

- The primary key, candidate keys and foreign keys can be specified as parts of the **create table** statement:
 - the **primary key** clause lists attributes that comprise the primary key.
 - the **unique key** clause lists attributes that comprise a candidate key.
 - the **foreign key** clause lists the attributes that comprise the foreign key and the name of the relation referenced by the foreign key.
 - ✓ a **foreign key** references the **primary key** attributes of the referenced table.

Cascading Actions in Referential Integrity

- **create table** *course* (
 course_id **char**(5) **primary key**,
 title **varchar**(20),
 dept_name **varchar**(20) **references** *department*)
- **create table** *course* (
 ...
 dept_name **varchar**(20),
 foreign key (*dept_name*) **references** *department*
 on delete cascade
 on update cascade,
 ...)
- alternative actions to cascade: **set null, set default**

Cascading Actions in Referential Integrity

- The **delete/update** operations on the **referenced** *department* will result in the **delete/update** on the **referencing** *course*

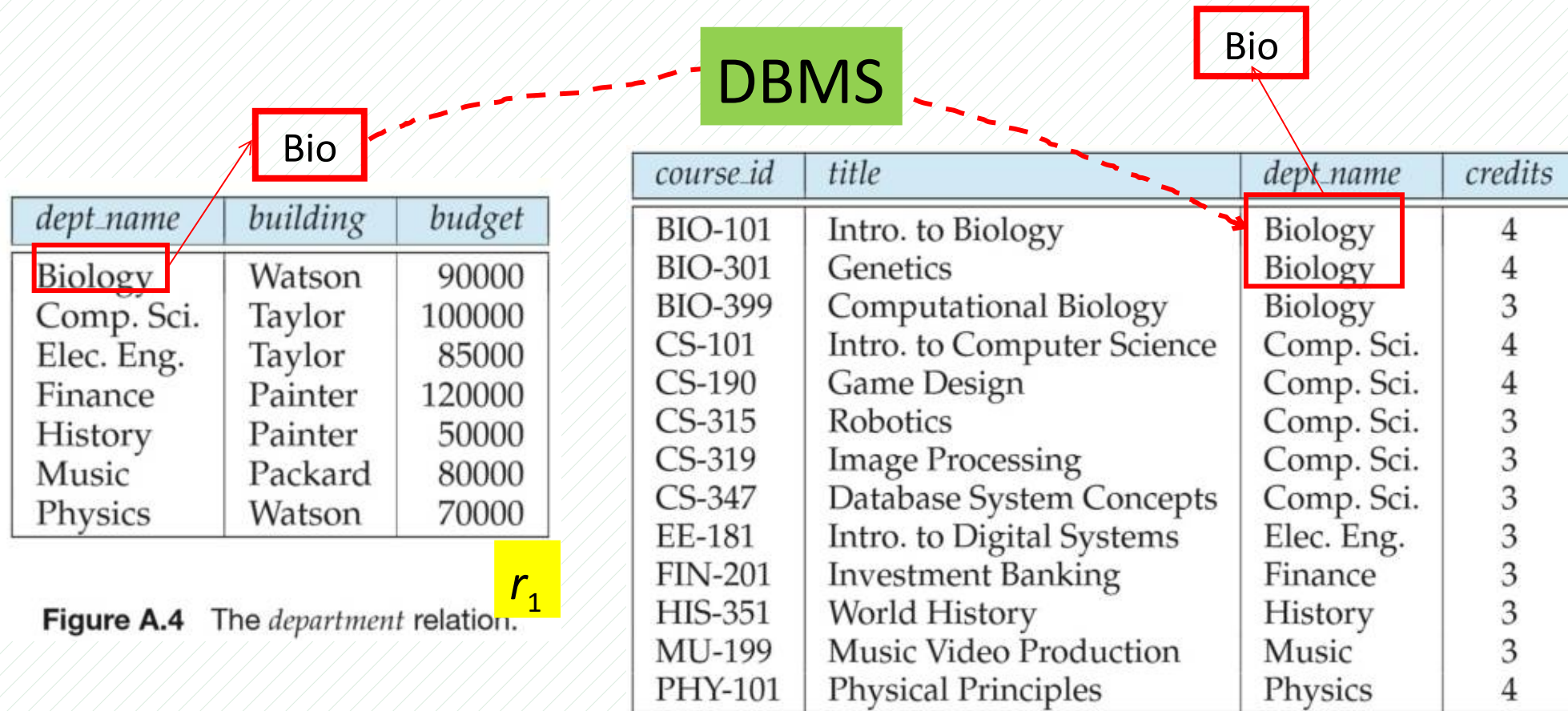


Figure A.4 The *department* relation.

Figure A.5 The *course* relation.

Integrity Constraint Violation During Transactions

- e.g create table *person* (
 ID char(10),
 name char(40),
 mother char(10),
 father char(10),
 primary key *ID*,
 foreign key *father* **references** *father*,
 foreign key *mother* **references** *mother*)
- How to insert a tuple without causing constraint violation ?
 - insert *father* and *mother* of a person **before** inserting *person*
 - OR, set father and mother to **null** initially, update **after** inserting all persons
(not possible if father and mother attributes declared to be **not null**)

如何在有主键 - 外键关联的两表上 modify 数据

4.5 Data Types and Schemas

■ built-in data types

- **date**: Dates, containing a (4 digit) year, month and date
 - e.g. date '2017-7-27'
- **time**: Time of day, in hours, minutes and seconds
 - e.g. time '09:00:30.75'
- **timestamp**: date plus time of day
 - e.g. timestamp '2017-7-27 09:00:30.75'
- **interval**: period of time
 - if x and y are of type date, then $x-y$ is an interval whose value is the number of days from date x to date y
 - e.g. interval 1 day='2017-7-27' – '2017-7-28'

Type Conversion

- **cast** *e* as *t*

- convert a **character string** *e* to the type *t*,
- e.g. **cast** '2017-07-20' as **date**
- e.g. **cast** '09:00:30.75' as **time**

```
select cast (ID as numeric(5)) as inst_id
  from   instructor
 order by inst_id
```

- **extract** value *d* from day or time

- For **day** or **time** value *d*, its individual fields can be **extracted**

- e.g. **extract** (**year from** 2017-07-20) =2017
- e.g. **extract** (**timezone_hour from** 2017-7-27 09:00:30.75) =09

CAST/CONVERT for Data Type Transition

CAST 和 CONVERT (Transact-SQL)

全部折叠 语言筛选器: 全部

将一种数据类型的表达式显式转换为另一种数据类型的表达式。CAST 和 CONVERT 提供相似的功能。

 [Transact-SQL 语法规则](#)

语法

Syntax for CAST:

```
CAST ( expression AS data_type [ (length) ] )
```

Syntax for CONVERT:

```
CONVERT ( data_type [ (length) ] , expression [ , style ] )
```

备注

隐式转换指那些没有指定 CAST 或 CONVERT 函数的转换。显式转换指那些需要指定 CAST 或 CONVERT 函数的转换。以下图例显示了可对数据类型转换。其中包括 xml、bigint 和 sql_variant。不存在对 sql_variant 数据类型的赋值进行的隐式转换，但是存在转换为 sql_variant 的

A. 同时使用 CAST 和 CONVERT

每个示例都检索列表价格的第一位是 3 的产品的名称，并将 ListPrice 转换为 int。

```
-- Use CAST
USE AdventureWorks;
GO
SELECT SUBSTRING(Name, 1, 30) AS ProductName, ListPrice
FROM Production.Product
WHERE CAST(ListPrice AS int) LIKE '3%';
GO
```

varchar, e.g. 350.00

```
-- Use CONVERT.
USE AdventureWorks;
GO
SELECT SUBSTRING(Name, 1, 30) AS ProductName, ListPrice
FROM Production.Product
WHERE CONVERT(int, ListPrice) LIKE '3%';
GO
```

B. 使用包含算术运算符的 CAST

以下示例将本年度截止到现在的全部销售额 (SalesYTD) 除以佣金百分比 (CommissionPCT), 从而得出单列计算结果 (Computed)。在舍入到最接近的整数后, 将此结果转换为 int 数据类型。



```
USE AdventureWorks;
GO
SELECT CAST(ROUND(SalesYTD/CommissionPCT, 0) AS int) AS 'Computed'
FROM Sales.SalesPerson
WHERE CommissionPCT != 0;
GO
```

C. 使用 CAST 进行连接

以下示例使用 CAST 连接非字符型非二进制表达式。

```
USE AdventureWorks;  
GO  
SELECT 'The list price is ' + CAST(ListPrice AS varchar(12)) AS ListPrice  
FROM Production.Product  
WHERE ListPrice BETWEEN 350.00 AND 400.00;  
GO
```



float

D. 使用 CAST 生成可读性更高的文本

以下示例使用选择列表中的 CAST 将 Name 列转换为 char(10) 列。

```
USE AdventureWorks;
GO
SELECT DISTINCT CAST(p.Name AS char(10)) AS Name, s.UnitPrice
FROM Sales.SalesOrderDetail s JOIN Production.Product p on s.ProductID = p.ProductID
WHERE Name LIKE 'Long-Sleeve Logo Jersey, M';
GO
```

varchar

Default Values

- **create table** *student*
(*ID* **varchar** (5),
name **varchar** (20) **not null**,
dept_name **varchar** (20),
tot_cred **numeric** (3,0) **default 0**,
primary key (*ID*))
- **insert into** *student*(*ID*, *name*, *dept_name*)
values ('12789', 'Newman', 'Comp.Sci')

只有 3
列

User-defined Types

- The **create type** clause can be used for user-defined types

- e.g. **create type** *Dollars* **as numeric (12,2)**

```
create table department  
  (dept_name varchar (20),  
   building varchar (15),  
   budget Dollars);
```

- **cast**, i.e. *convert*, values of one type to another domain

- **cast** (*department.budgt* to *numeric(12.2)*),

then the expression

(department.budgt + 20)

can be evaluated

User-defined Domains

- **create domain** construct creates **user-defined domain** types

- **create domain** *person_name* **char**(20) **not null**

- **Types and domains** are similar.

Domains can have constraints, such as **not null**, specified on them.

- **create domain** *degree_level* **varchar**(10)
constraint *degree_level_test*
check (**value in** ('Bachelors', 'Masters', 'Doctorate'));

实验要求

- 以 openGauss/ 为平台，定义 2 张表间的无级联 modification、有级联 modification 的外键关联，
判断：当增 / 删 / 改一张表中的主键、外键值时，DBMS 的反应

数据库系统设计时，是否使用外键？！

- 外键反映了不同数据项间的逻辑关联和约束关系，是一种客观存在
 - e.g. *dept_name* in the table *department*, *course*
- 对外键的处理方式反映了对这种数据相互间关系的处理态度，需要根据实际情况而定，考虑数据完整性、处理效率等多种因素
 - 并非简单的“需要外键”，“不设外键”（ e.g. 部分 MySQL 课程、 MySQL 社区的论点 ）
- **方式 1**：设置数据库表的外键
 - 优点
 - DBMS 自动维护数据间的关联、约束关系
 - 缺点
 - 数据导入、增删改时，多个表间的相互关联，保证了数据一致性、完整性，但影响到整体数据处理效率（ 额外的 DBMS 处理开销 ）
——e.g. 插入一条 *dept_name* 不存在的 *course* 元组，失败，并影响前面已经成功插入的数据——局部错误影响整体

数据库系统设计时，是否使用外键？！

- **方式 2**：不设置外键，由应用程序在业务逻辑层维护数据一致性、完整性
 - 优点
 - 1) 避免局部错误影响整体进度；
 - 2) 业务逻辑层的数据一致性处理的速度可以比较快， e.g. 将数据提至内存中批量处理
 - 缺点：对应用层业务处理程序设计的额外要求
- 上述 2 种方式用在对数据一致性完整性要求高的场景下， e.g. 银行
- **方式 3**：不设置外键，业务层应用逻辑也不处理数据间一致性
 - 牺牲数据的正确性，换取数据处理的效率
 - e.g. 大数据应用场景下，对数据精度、正确性要求不高

LTE 网络数据库课程实验：数据库系统设计时，是否使用外键？

- tbMRData、tbATUData 表中的主小区、邻小区 ID 对应于 tbCell 中的小区标识 Sector_ID
- 实际应用中，不要在 tbMRData、tbATUData 与 tbCell 中间建立外键。否则，两表间由于强行的外键约束，可能带来低效的数据处理
 - ▣ E.g. 1 向 tbMRData、tbATUData 中大批量导入数据时，插入的某条测量报告数据中的某些主小区、邻小区没有出现在 tbCell 表中，导致本条以及前面各条已成功插入的测量报告均插入失败
 - ▣ E.g.2 当 tbCell 中某个 cell/sector 修改名字时，在（非常非常大的，e.g. T 级别，上亿条）tbMRData、tbATUData 表中以该小区作为主、邻小区的测量报告也必须随之修改
- 现网环境中，tbMRData、tbATUData 的数据与 tbCell 的数据在主小区、邻小区属性上经常存在不一致，无法保证外键关联约束，原因：
 - ▣ tbMRData、tbATUData 中的测量报告的测量范围超出 tbCell 表范围，有些测量报告的主小区、邻小区没有出现在 tbCell 表中
 - ▣ tbCell 表中的小区数据有可能被修改，导致与 tbMRData、tbATUData 中的测量报告不一致

实验要求：LTE 网络数据库

- 在建表、数据导入阶段，先不要建立 tbMRData、tbATUData 与 tbCell 间的外键
- 数据导入完成后，再建立表间外键关联
观察：
 - ▣ 是否能建立？
 - ▣ 如果建立，tbCell 中主键值的变化对 tbMRData、tbATUData 的影响，或反之

在已经建立的表中增加外键的方法，参见具体 DBMS 平台用户手册

Complex Check Conditions and Assertions

- Some constructs such as ***check*** and ***assertion*** are defined to specify the complex integrity constraints.
 - **however, they are not currently supported by most database systems**
- e.g. For the relation ***section***, the `time_slot_id` in each tuple should be is actually the identifier of a `time_slot` in the ***time_slot*** relation. This constraint can be defined as
check (time_slot_id in (select time_slot_id from time_slot))
- ***check*** in *create table*

```
create table student (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    tot_cred    numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department)  
    check(tot_cred>0)  
)
```



Assertions

- An *assertion* is a predicate expressing a condition that we wish the database always to satisfy
 - e.g. domain constraints, referential-integrity constraint
- An assertion takes the form
create assertion <assertion-name> **check** <predicate>
 - also **not** supported by anyone, e.g. SQL Server 等数据库系统似乎不支持创建
- Why ?
 - When an assertion is made, the DBMS tests it for validity.
Any modification to DB is allowed only if it does not cause that assertion to be violated
 - this testing may introduce a significant amount of overhead,
hence assertions should be used with great care

Assertion Example

- For each tuple in the *student* relation, the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully



- **create assertion** *credits_earned_constraints* **check**
 (not exists (select *ID*
 from *student*
 where *tot_cred*
 <> **select** **sum**(*credits*)
 from *takes* **natural join** *course*
 where *student.ID=takes.ID*
 and *grade* **is not null**
 and *grade*<>'F')
)

4.6 Index Definition in SQL

- Many queries reference **only a small proportion** of the records in a table.
- It is **inefficient** to read every record to find a record with particular value
- An **index** on an attribute of a relation is a data structure that allows the DBS to find those tuples in the relation that have a specified value for that attribute efficiently, **without scanning through all the tuples of the relation**
- **Create** an index with the **create index** command
create index <name> **on** <relation-name> (attribute);

Index Definition in SQL

- **create table** *student*
(*ID* **varchar** (5),
name **varchar** (20) **not null**,
dept_name **varchar** (20),
tot_cred **numeric** (3,0) **default** 0,
primary key (*ID*))
- **create index** *studentID_index* **on** *student*(*ID*)
- The query:
 select *
 from *student*
 where *ID* = '12345'

can be executed by using the **index** to find the required record,
without looking at all records of *student*

4.7 Authorization

- Forms of authorization on parts of the *database*
 - **Read** - allows reading, but not modification of data
 - **Insert** - allows insertion of new data, but not modification of existing data
 - **Update** - allows modification, but not deletion of data
 - **Delete** - allows deletion of data
- Forms of authorization to modify the *database schema* (Chapter 8)
 - **resources** - allows creation of new relations
 - **alteration** - allows addition or deletion of attributes in a relation
 - **drop** - allows deletion of relations
 - **index**

Authorization Specification

- The **grant** statement is used to confer authorization
 - grant** <privilege list>
on <relation name or view name> **to** <user list>
 - <user list> is:
 - ▣ a user-id
 - ▣ **public**, which allows all valid users the privilege granted
- Granting a privilege on a **view** does **not** imply granting any privileges on the underlying relations
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator)

Privileges

- **select**: allows **read access** to relation, or **ability to query** using the view
 - Example: grant users U_1 , U_2 , and U_3 **select** authorization on *instructor* relation:
grant select on *instructor* to U_1 , U_2 , U_3
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

Revoking Authorization

- The **revoke** statement is used to revoke authorization.
revoke <privilege list>
on <relation name or view name> **from** <user list>
- Example:
revoke select on branch from U_1 , U_2 , U_3
- <privilege-list> may be **all** to revoke all privileges the revoker may hold.
- If <revokee-list> includes **public**, all users lose the privilege except those granted it explicitly

Roles

- **create role** instructor;
- **grant** *instructor* **to Amit**;
- Privileges can be granted to roles:
 - **grant** **select** **on** *takes* **to instructor**;
- Roles can be granted to users, as well as to other roles
 - **create role** *teaching_assistant*
 - **grant** *teaching_assistant* **to instructor**;
 - *Instructor* inherits all privileges of *teaching_assistant*
- Chain of roles
 - **create role** *dean*;
 - **grant** *instructor* **to dean**;
 - **grant** *dean* **to Satoshi**;

Authorization on Views

- **create view** *geo_instructor* as
(**select** *
from *instructor*
where *dept_name* = 'Geology');
- **grant** **select** **on** *geo_instructor* **to** *geo_staff*
- Suppose that a *geo_staff* member issues
 - ▣ **select** *
from *geo_instructor*;
- What if
 - ▣ *geo_staff* does not have permissions on *instructor*?
 - ▣ creator of view did not have some permissions on *instructor*?

Other Authorization Features

- **references** privilege to create foreign key
 - ▣ **grant reference** (*dept_name*) **on** *department* **to** Mariano;
 - ▣ why is this required?
- **transfer of privileges**
 - ▣ **grant select on** *department* **to** Amit **with grant option**;
 - ▣ **revoke select on** *department* **from** Amit, Satoshi **cascade**;
 - ▣ **revoke select on** *department* **from** Amit, Satoshi **restrict**;
- Etc. read Section 4.6 for more details we have omitted here

需结合具体 DBMS 平台

Appendix 4-1 *create table* in T-SQL

```
CREATE TABLE
    [ database_name . [ schema_name ] . | schema_name . ] table_name
    ( { <column_definition> | <computed_column_definition> }
      [ <table_constraint> ] [ ,...n ] )
    [ ON { partition_scheme_name ( partition_column_name ) | filegroup
      | "default" } ]
    [ { TEXTIMAGE_ON { filegroup | "default" } } ]
    [ ; ]
```

分区

文件组

物理设计

```
<column_definition> ::=
column_name <data_type>
    [ COLLATE collation_name ]
    [ NULL | NOT NULL ]
    [
        [ CONSTRAINT constraint_name ] DEFAULT constant_expression ]
    | [ IDENTITY [ ( seed ,increment ) ] [ NOT FOR REPLICATION ]
    ]
    [ ROWGUIDCOL ] [ <column_constraint> [ ...n ] ]
```

```

<column_constraint> ::=
[ CONSTRAINT constraint_name ]
{
    { PRIMARY KEY | UNIQUE }
    [ CLUSTERED | NONCLUSTERED ]
    [
        WITH FILLFACTOR = fillfactor
        | WITH ( < index_option > [ , ...n ] )
    ]
}

```

聚集 / 非聚集索引
(物理设计)

外键

```

]
[ ON { partition_scheme_name ( partition_column_name )
    | filegroup | "default" } ]
[ FOREIGN KEY ]
REFERENCES [ schema_name . ] referenced_table_name [ ( ref_column ) ]
[ ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
[ ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]
[ NOT FOR REPLICATION ]
| CHECK [ NOT FOR REPLICATION ] ( logical_expression )
}

```

```
<computed_column_definition> ::=  
column_name AS computed_column_expression  
[ PERSISTED [ NOT NULL ] ]  
[  
    [ CONSTRAINT constraint_name ]  
    { PRIMARY KEY | UNIQUE }  
    [ CLUSTERED | NONCLUSTERED ]  
    [  
        WITH FILLFACTOR = fillfactor  
        | WITH ( <index_option> [ , ...n ] )  
    ]  
    | [ FOREIGN KEY ]  
        REFERENCES referenced_table_name [ ( ref_column ) ]  
        [ ON DELETE { NO ACTION | CASCADE } ]  
        [ ON UPDATE { NO ACTION } ]  
        [ NOT FOR REPLICATION ]  
    | CHECK [ NOT FOR REPLICATION ] ( logical_expression )  
    | ON { partition_scheme_name ( partition_column_name )  
        | filegroup | "default" } ]  
]
```

派生属性

```
< table_constraint > ::=  
[ CONSTRAINT constraint_name ]  
{  
    { PRIMARY KEY | UNIQUE }  
    [ CLUSTERED | NONCLUSTERED ]
```

```
        (column [ ASC | DESC ] [ ,...n ] )  
    [  
        WITH FILLFACTOR = fillfactor  
        | WITH ( <index_option> [ , ...n ] )  
    ]  
    [ ON { partition_scheme_name (partition_column_name)  
        | filegroup | "default" } ]  
    | FOREIGN KEY  
        ( column [ ,...n ] )  
        REFERENCES referenced_table_name [ ( ref_column [ ,...n ] ) ]  
        [ ON DELETE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]  
        [ ON UPDATE { NO ACTION | CASCADE | SET NULL | SET DEFAULT } ]  
        [ NOT FOR REPLICATION ]  
    | CHECK [ NOT FOR REPLICATION ] ( logical_expression )  
}
```



```

<index_option> ::=
{
    PAD_INDEX = { ON | OFF }
    | FILLFACTOR = fillfactor
    | IGNORE_DUP_KEY = { ON | OFF }
    | STATISTICS_NORECOMPUTE = { ON | OFF }
    | ALLOW_ROW_LOCKS = { ON | OFF }
    | ALLOW_PAGE_LOCKS = { ON | OFF }
}

```

备注

在 SQL Server 2005 中，每个数据库最多可包含 20 亿个表，每个表可包含 1,024 列。表的行数及总大小仅受可用存储空间的限制。每行最多包括 8,060 个字节。对于带 varchar、nvarchar、varbinary 或 sql_variant 列（导致已定义表的总宽超过 8,060 字节）的表，此限制将放宽。其中每列的长度仍必须在 8,000 字节的限制内，但是它们的总宽可能超过表的 8,060 字节的限制。有关详细信息，请参阅[行溢出数据超过 8 KB](#)。

每个表最多可以有 249 个非聚集索引和 1 个聚集索引。其中包括为支持表中所定义的 PRIMARY KEY 和 UNIQUE 约束而生成的索引。

通常情况下，为表和索引分配空间时，每次以一个区为增量单位。当创建表或索引时，首先从混合区为其分配页，直到它具有足够的页填满一个统一区。当足够的页填满统一区后，每当当前分配的区域填满时，将再为其分配另一个区。若要获得关于由表分配和占用的空间量的报表，请执行 sp_spaceused。

数据库引擎 在列定义中并不强制以特定的顺序指定 DEFAULT、IDENTITY、ROWGUIDCOL 或列约束。

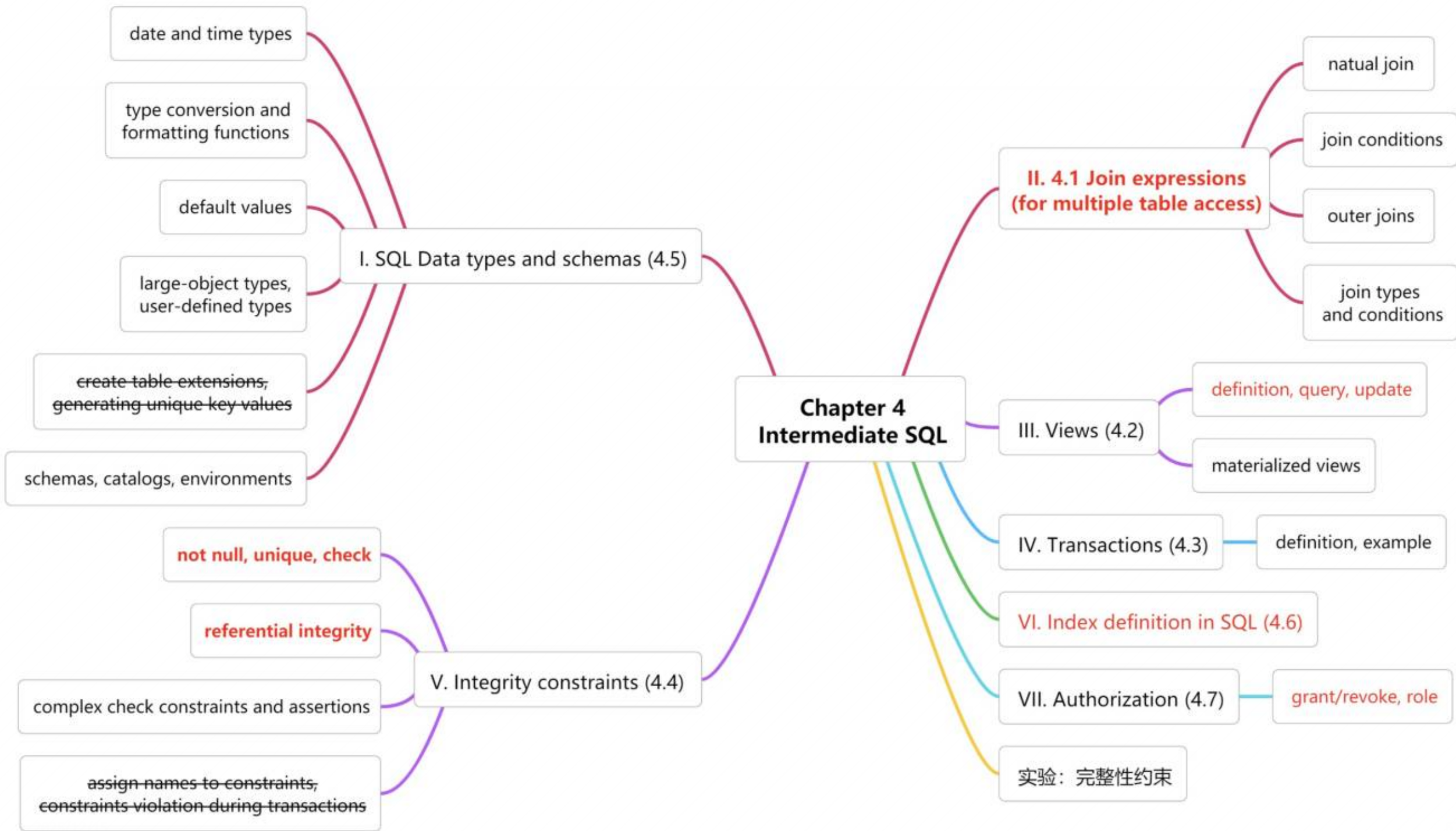
创建表后，即使 QUOTED IDENTIFIER 选项在创建表时设置为 OFF，该选项在表的元数据中仍存储为 ON。

临时表

可以创建本地临时表和全局临时表。本地临时表仅在当前会话中可见，而全局临时表在所有会话中都可见。临时表不能分区。

本地临时表的名称前面有一个数字符号 (#table_name)，而全局临时表的名称前面有两个数字符号 (##table_name)。

SQL 语句使用 CREATE TABLE 语句中为 table_name 指定的值引用临时表，例如：





Thanks for your
attention